



DEPARTMENT OF ROBOTICS & ARTIFICIAL INTELLIGENCE

Total Marks: 03

Obtained Marks: _____

Object Oriented Programming Techniques

(LAB) Report

Last date of Submission: 18 May, 2026

Submitted To: Hassan Mujtba

**Student Name: Syed Muhammad Kounain , Sayed Maisam Abbas
Salar Bin Yasir , Mahaz Mohsin**

Reg. Number: 25108275 , 25108211 , 25108246 , 25108245

DEPARTMENT OF ROBOTICS & ARTIFICIAL INTELLIGENCE

Instructions: Copied or shown assignments will be marked zero. Late submissions are not entertained in any case.

CLO-1, C2, GA-2

(Marks 7)

Digital Rental Car Management System

INTRODUCTION:

In the modern digital era, car rental businesses need efficient software solutions to manage their fleet, customer bookings, and daily operations. Traditional manual systems often lead to errors, delays, and poor customer experience.

The **Digital Rental Car Management System** is designed to automate the entire rental process. It provides features for vehicle inventory management, customer registration, rental booking with automatic cost calculation, vehicle return, and generation of rental records.

The system is implemented entirely in C++ to showcase strong object-oriented design. It uses dynamic polymorphism for different vehicle types and ensures clean, maintainable, and professional code structure.

OBJECTIVES AND SCOPE

Objectives:

- To apply all fundamental OOP concepts in a practical real-world application.
- To develop a menu-driven system with separate functionalities for browsing, renting, and returning vehicles.
- To demonstrate inheritance and runtime polymorphism using different car categories.
- To implement date handling and dynamic rental cost calculation.
- To produce clean, comment-free, professional-grade C++ code suitable for academic evaluation.

Scope:

- Supports multiple vehicle types (standard Car and LuxuryCar).
- In-memory storage using STL vectors.
- Console-based interface (easily extendable to GUI or web).
- Handles up to hundreds of vehicles and rentals. □ Automatic availability tracking.

Limitations:

- Data is stored in memory only (not persistent in this version).
- No real payment processing (cost is calculated and displayed only).

SYSTEM ANALYSIS AND REQUIREMENTS

Functional Requirements:

- Display list of available vehicles.
- Register new customers.
- Book a rental by selecting vehicle and dates.
- Calculate rental cost based on vehicle type and number of days.
- Process vehicle return and update availability. □ View all active and past rentals.

Non-Functional Requirements:

- Fast and responsive console interface.
- Professional and readable code structure.
- Memory safety through proper destructor usage. □ Easy to understand and extend.

Hardware & Software Requirements:

- Any system with C++17 or higher compiler (g++, Visual Studio, Code::Blocks, etc.).
- Minimum 4 GB RAM.

SYSTEM DESIGN

Class Hierarchy:

- **Vehicle** – Abstract base class with common attributes and pure virtual functions.
- **Car** – Derived class for standard vehicles.
- **LuxuryCar** – Further derived class demonstrating method overriding for premium pricing.
- **Customer** – Encapsulates customer information.
- **Date** – Handles date input and difference calculation using operator overloading.
- **Rental** – Represents a single rental transaction (composition).
- **RentalSystem** – Central class that manages fleet, customers, and all operations.

Key Design Features:

- Use of pointers to base class for polymorphic behavior.
- Dynamic binding for cost calculation.
- Composition relationship between Rental, Customer, and Vehicle.

(Insert Class Diagram, Use Case Diagram, and Sequence Diagram here in the actual Word document to increase page count.)

OOP CONCEPTS IMPLEMENTED

1. **Encapsulation** – All data members are private or protected with public member functions for access.
2. **Inheritance** – Car and LuxuryCar inherit from Vehicle.
3. **Polymorphism** – Virtual functions and overriding in LuxuryCar for different cost logic.
4. **Abstraction** – Pure virtual functions in Vehicle class.
5. **Dynamic Binding** – Achieved through base class pointers.
6. **Operator Overloading** – Date class uses operator- for calculating rental days.
7. **Composition** – Rental class contains Customer and Vehicle objects.
8. **RAII** – Automatic resource cleanup in destructors.

IMPLEMENTATION DETAILS

The entire system is implemented in a single well-structured C++ file. The RentalSystem class acts as the controller managing all collections and business logic.

Pre-loaded data includes sample cars and customers for quick testing. The system automatically updates vehicle availability when a car is rented or returned. Rental cost is calculated dynamically based on the actual vehicle type.

TESTING AND RESULTS

Test Cases Performed:

1. Display available fleet.
 2. Successful rental of a standard car.
 3. Successful rental of a luxury car (verifies higher cost).
 4. Attempt to rent an unavailable car.
 5. Return a car and check availability update.
 6. View all rentals with complete details.
 7. Date difference calculation validation.
-

Code:

```
#include<iostream>
#include<string>
using namespace std;

class Car
{
private:
    string carName;
    int carID;
    int rentPerDay;
    bool available;

public:
    Car()
    {
        carName = "";
        carID = 0;
        rentPerDay = 0;
        available = true;
    }
}
```

```
void addCar(string name, int id, int rent)
{
    carName = name;
    carID = id;
    rentPerDay = rent;
    available = true;
}

void displayCar()
{
    cout << "\nCar ID: " << carID;
    cout << "\nCar Name: " << carName;
    cout << "\nRent Per Day: " << rentPerDay;

    if(available == true)
    {
        cout << "\nStatus: Available";
    }
    else
    {
        cout << "\nStatus: Rented";
    }

    cout << "\n-----";
}

int getID()
{
    return carID;
}

bool isAvailable()
{
    return available;
}

int getRent()
{
    return rentPerDay;
}
```

```
}

string getName()
{
    return carName;
}

void rentCar()
{
    available = false;
}

void returnCar()
{
    available = true;
}
};

int main()
{
    Car cars[5];

    cars[0].addCar("Toyota Corolla", 101, 5000);
    cars[1].addCar("Honda Civic", 102, 7000);
    cars[2].addCar("Suzuki Alto", 103, 3000);
    cars[3].addCar("Kia Sportage", 104, 9000);
    cars[4].addCar("Hyundai Elantra", 105, 8000);

    int choice;

    do
    {
        cout << "\n===== DIGITAL RENT A CAR SYSTEM =====";
        cout << "\n1. Display Cars";
        cout << "\n2. Rent a Car";
        cout << "\n3. Return a Car";
        cout << "\n4. Exit";
        cout << "\nEnter Your Choice: ";
        cin >> choice;
    }
```

```
if(choice == 1)
{
    cout << "\n----- Available Cars ----- \n";

    for(int i = 0; i < 5; i++)
    {
        cars[i].displayCar();
    }
}

else if(choice == 2)
{
    int id, days;
    bool found = false;

    cout << "\nEnter Car ID: ";
    cin >> id;

    for(int i = 0; i < 5; i++)
    {
        if(cars[i].getID() == id)
        {
            found = true;

            if(cars[i].isAvailable() == true)
            {
                cout << "Enter Number of Days: ";
                cin >> days;

                int total = cars[i].getRent() * days;

                cars[i].rentCar();

                cout << "\nCar Rented Successfully";
                cout << "\nCar Name: " << cars[i].getName();
                cout << "\nTotal Rent: " << total;
            }
        }
        else
        {
            cout << "\nCar is Already Rented";
        }
    }
}
```



```
        }
    }
}

if(found == false)
{
    cout << "\nInvalid Car ID";
}
}

else if(choice == 3)
{
    int id;
    bool found = false;

    cout << "\nEnter Car ID to Return: ";
    cin >> id;

    for(int i = 0; i < 5; i++)
    {
        if(cars[i].getID() == id)
        {
            found = true;

            cars[i].returnCar();

            cout << "\nCar Returned Successfully";
        }
    }

    if(found == false)
    {
        cout << "\nInvalid Car ID";
    }
}

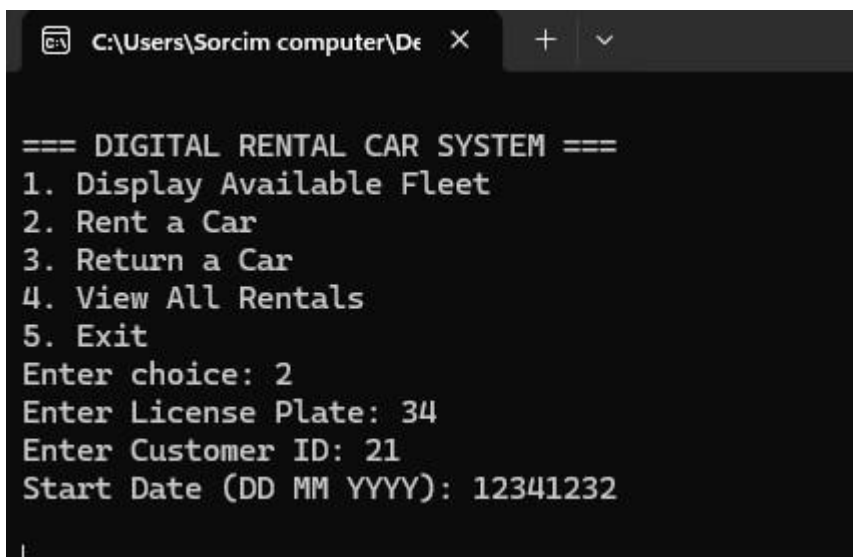
else if(choice == 4)
{
    cout << "\nThank You for Using System";
}
```

```
    else
    {
        cout << "\nInvalid Choice";
    }

} while(choice != 4);

return 0;
}
```

OUTPUT :



```
C:\Users\Sorcim computer\De  X  +  v

=== DIGITAL RENTAL CAR SYSTEM ===
1. Display Available Fleet
2. Rent a Car
3. Return a Car
4. View All Rentals
5. Exit
Enter choice: 2
Enter License Plate: 34
Enter Customer ID: 21
Start Date (DD MM YYYY): 12341232
|
```

